The Bayesian Orchestrator

Alisa Sheinkman

June 16, 2026

This work proposes placing Bayesian inference in the orchestration layer of AI agents and systems. Such orchestrator estimates uncertainty of the model reliability from observed outcomes, then choose calls by expected utility. This follows the control-layer argument of Papamarkou et al. [1] and avoids assuming that an LLM performs coherent internal Bayesian updating.

1 Decision Problem

For question q and candidate model m, let $y_{q,m}$ indicate correctness, $c_{q,m}$ denote call cost, and λ denote the configured cost scale:

$$y_{q,m} \in \{0,1\}, \qquad c_{q,m} \geq 0, \qquad \lambda \geq 0.$$

The realized utility of a call is:

$$u(y_{q,m}, c_{q,m}) = y_{q,m} - \lambda c_{q,m}.$$

Given data for exploration D, the posterior predictive probability and expected utility follow the standard Bayesian statistical decision-theory formulation [2]:

$$p_{q,m} = \mathbb{P}(y_{q,m} = 1 \mid D), \quad \mathbb{E}[u \mid D, q, m] = p_{q,m} - \lambda c_{q,m}.$$

Then the single-shot policy chooses:

$$\pi_{\text{single}}(q) = \arg\max_{m} \left\{ p_{q,m} - \lambda c_{q,m} \right\}.$$

2 Exploration Data and Call Matrix

For every selected question and model, the workflow records the question, answer, confidence, latency, token counts, cost, parsing status, and correctness. The resulting full call matrix is an evaluation baseline, it allows for exact offline policy comparison on the sampled questions. Questions are divided into exploration and evaluation sets, that is for $Q_{\text{explore}} \cap Q_{\text{test}} = \varnothing$, define

$$D_{\text{train}} = \{(q, m, \ldots, y_{q,m}) : q \in Q_{\text{explore}}\}, \quad D_{\text{test}} = \{(q, m, \ldots, y_{q,m}) : q \in Q_{\text{test}}\},$$

During simulated policy execution a policy may inspect only outputs from calls it selected. Uncalled held-out responses remain hidden until scoring.

3 Hierarchical Bayesian Reliability Model

Each row i of a call matrix represents one question-model call. Its pre-call features are model identity, subject, question-length bucket, and configured cost weight. Post-call features additionally include parsed confidence, latency, token count, and observed token-price cost. Correctness follows a Bernoulli-logistic hierarchical model, with partial pooling and prior specification grounded in standard Bayesian modeling practice [3]:

$$y_i \sim \mathrm{Bernoulli}(\sigma(\eta_i)), \qquad \sigma(x) = \frac{1}{1+\exp(-x)}.$$

The pre-call predictor contains only information available before a response is observed:

$$\eta_i^{\mathrm{pre}} = \alpha + a_{m_i}^{\mathrm{model}} + a_{s_i}^{\mathrm{subject}} + a_{\ell_i}^{\mathrm{length}} + \beta_{\mathrm{cost}} w_i.$$

The post-call model adds observed response features:

$$\begin{aligned}\eta_i^{\mathrm{post}} = \eta_i^{\mathrm{pre}} &+ \beta_{\mathrm{latency}} \widetilde{t}_i + \beta_{\mathrm{tokens}} \widetilde{n}_i \\ &+ \beta_{\mathrm{observed\ cost}} \widetilde{c}_i + \beta_{\mathrm{confidence}}(r_i - 0.5).\end{aligned}$$

Here, r_i is parsed self-confidence, $\widetilde{t}_i$ is transformed latency, $\widetilde{n}_i$ is transformed token count, and $\widetilde{c}_i$ is observed token-price cost when available.

The priors set for the MMLU-Pro evaluation are:

$$\begin{aligned}\alpha &\sim \mathcal{N}(0, 1.5^2), \\ \sigma_{\mathrm{model}} &\sim \mathrm{HalfNormal}(1), \\ \sigma_{\mathrm{subject}} &\sim \mathrm{HalfNormal}(1), \\ a_m^{\mathrm{model}} &\sim \mathcal{N}(0, \sigma_{\mathrm{model}}^2), \\ a_s^{\mathrm{subject}} &\sim \mathcal{N}(0, \sigma_{\mathrm{subject}}^2), \\ a_\ell^{\mathrm{length}} &\sim \mathcal{N}(0, 0.5^2), \\ \beta_j &\sim \mathcal{N}(0, 1).\end{aligned}$$

The posterior is sampled with HMC in NumPyro. For posterior draws $\theta^{(1)}, \ldots, \theta^{(S)}$, the Monte Carlo approximation of predictive reliability is:

$$\mathbb{P}(y_i = 1 \mid D) \approx \frac{1}{S} \sum_{s=1}^{S} \sigma\left(\eta_i(\theta^{(s)})\right).$$

Pre-call probabilities determine the first LLM that router decides to query.

Post-call probabilities, when high, support the orgestration stoping and returning the answer, or, when post-call probabilities are low, the orchestrator calculates if it is worth to call a second model and combine the answers.

4 Adaptive Calls

The adaptive policy uses a myopic decision-value approximation inspired by the value-of-information framework [4] and adaptive information gathering [5]. The limitation of the approach is that it does not optimize the full expected information gain objective of sequential Bayesian experimental design. Let C be the set of models that were already called and let the current best post-call reliability be:

$$b = \max_{m \in C} p_{\text{post}}(q, m).$$

For an uncalled model m, let V_m be the empirical distribution of post-call reliability values observed in matching exploration groups. The estimated myopic gain is:

$$\Delta(q, m) = \mathbb{E}_{v \sim V_m}[\max(b, v)] - b - \lambda \mathbb{E}[c_m].$$

The configuration in experiments uses the adjudication strategy or, in other words, Noisy-OR rule [6] applied in the case of LLM ensembling. For an answer option a, the aggregation assumes that reliability signals are independant and supporting calls are combined using score defined as:

$$1 - \prod_{m \in C:\, \text{answer}_m = a} \left(1 - p_{\text{post}}(q, m)\right).$$

Let s_C denote the current adjudicated answer score and let a particular uncalled candidate m have the pre-call correctness probability $p_{\text{pre}}(q, m)$. Using the formula for myopic gain and the Noisy-OR aggregation, the expected utility gain from calling m is:

$$\Delta_{\text{adj}}(q, m) = (1 - s_C) p_{\text{pre}}(q, m) - \lambda c_m.$$

A second call is accepted when:

$$\max_m \Delta_{\text{adj}}(q, m) > \tau_{\text{gain}},$$

where τ_{gain} acts as a minimum threshold for expected utility gain. The above is subject to invalid first answer activates the explicit fallback rule and to the configured maximum number of calls.

5 Offline Policy Values

For deterministic policy π, held-out empirical utility is:

$$\widehat{V}(\pi) = \frac{1}{|Q_{\text{test}}|} \sum_{q \in Q_{\text{test}}} \left[y_{q,\pi(q)} - \lambda c_{q,\pi(q)} \right].$$

Uniform random routing has value:

$$\widehat{V}_{\text{random}} = \frac{1}{|Q_{\text{test}}|} \sum_{q \in Q_{\text{test}}} \frac{1}{M} \sum_{m=1}^{M} \left[y_{q,m} - \lambda c_{q,m} \right].$$

The fixed-model baseline for model m is:

$$\widehat{V}_{\text{always}\, m} = \frac{1}{|Q_{\text{test}}|} \sum_{q \in Q_{\text{test}}} \left[y_{q,m} - \lambda c_{q,m} \right].$$

The adaptive policy pays for every call it makes:

$$u_{\text{adaptive}}(q) = \mathbf{1}\{\hat{a}_q = a_q^*\} - \lambda \sum_{m \in C_q} c_{q,m}.$$

The oracle retrospectively selects the best observed model:

$$\widehat{V}_{\text{oracle}} = \frac{1}{|Q_{\text{test}}|} \sum_{q \in Q_{\text{test}}} \max_m \left[y_{q,m} - \lambda c_{q,m} \right],$$

which, while not deployable, provides an upper bound for the sampled question-model matrix. Question-level bootstrap resampling estimates finite-sample uncertainty in policy differences such as:

$$\widehat{V}(\pi_{\text{adaptive}}) - \widehat{V}(\pi_{\text{random}}).$$

6 MMLU-Pro Results

The MMLU-Pro run used seed 48219, a proportional stratified sample of 4,000 MMLU-Pro test questions, and three LLM models: NVIDIA's Nemotron-3-Nano (30B), Qwen3 (30B as well but with higher costs), gpt-oss (120B). An obvious limitation of this evaluation is that the sizes of the small and middle models are similar, such choice have been primarily driven by the avaliability of models.

The reliability models were fit on 1,200 exploration questions and evaluated on 2,800 disjoint held-out questions. The call matrix contains all 12,000 question-model pairs. Prices use the Token Factory snapshot dated June 14, 2026. The calibrated configuration uses $\lambda = 200$ and $\tau_{\text{gain}} = 0.1$. For example, a call costing 0.001 receives a utility penalty of 0.2. Note that λ and τ_{gain} are chosen empirically, so they should be comfirmed before evaluation on a fresh confirmation sample.

Table 1 reports policy accuracy, observed cost, and utility on the held-out set.

Policy	Accuracy	USD / 1,000	Utility
Random model	0.605	0.1651	0.5725
Always small	0.519	0.1727	0.4847
Always middle	0.564	0.0313	0.5577
Always strong	0.733	0.2912	0.6750
Bayesian single shot	0.739	0.2788	0.6832
Adaptive Bayesian	**0.763**	0.2950	**0.7042**
Retrospective oracle	0.817	0.0871	0.7994

Table 1: Held-out results for the final run. The oracle uses retrospective knowledge and is not deployable.

The adaptive policy requested a second call for 33.4% of questions and averaged 1.334 calls per question. Compared with single-shot Bayesian routing, adaptive routing improved accuracy by 2.4 percentage points and utility by 0.0210, while observed token cost increased by approximately 5.8%. Compared with uniform random routing, the utility difference was 0.1318. Given this MMLU-PRO sample/split, model set, prompt, price snapshot, and utility scale, **Bayesian adaptive routing improved held-out performance**.

The workflow reports approximate leave-one-out expected log predictive density [7], Brier score [8], negative log-likelihood, expected calibration error [9], and AUROC. Self-reported confidence is treated as a feature to validate rather than a trustworthy posterior, consistent with evidence that elicited LLM beliefs can be behaviorally incoherent [10] and that LLM belief updates are not consistently Bayesian [11]. Function-calling uncertainty work further motivates auditing output validity and call-specific uncertainty signals [12]. Self-reported confidence improved correctness prediction as an input feature, but raw confidence was high across models and should not be interpreted as calibrated probability.

The oracle's low apparent cost is an artifact of retrospective selection: it can choose the cheapest successful observed call after seeing all outcomes. It is an upper bound for this complete matrix, not a feasible routing strategy.

7 Synthetic Router and a Toy Example

For synthetic task t assigned to agent a:

$$\text{success}_t \sim \text{Bernoulli}\left(\sigma\left(\alpha_a + \beta_a^{\text{difficulty}}\text{difficulty}_t + \beta_a^{\text{ambiguity}}\text{ambiguity}_t\right)\right).$$

The fitted priors are:

$$\begin{aligned}
\mu_{\text{agent}} &\sim \mathcal{N}(0, 1.5^2),\\
\tau_{\text{agent}} &\sim \text{HalfNormal}(1),\\
\alpha_a &\sim \mathcal{N}(\mu_{\text{agent}}, \tau_{\text{agent}}^2),\\
\beta_a^{\text{difficulty}} &\sim \mathcal{N}(-2, 1),\\
\beta_a^{\text{ambiguity}} &\sim \mathcal{N}(-0.75, 0.75^2).
\end{aligned}$$

For each held-out task, the selected agent is:

$$\arg\max_a \left\{\mathbb{P}(\text{success} = 1 \mid \text{task}, a, D) - c_a\right\}.$$

Because this workflow is simulated, true counterfactual expected utilities are available for every agent. Real routing logs require randomized exploration, logged propensities, controlled experiments, or valid off-policy estimators.

8 Limitations and Related Work

The implementation is a pragmatic Bayesian controller for model routing. It does not claim that the underlying LLMs are Bayesian, that elicited confidence is intrinsically calibrated, or that the adaptive policy implements full Bayesian experimental design. Conclusions remain specific to the selected questions, model versions, prompts, prices, and run date.

BayesAgent provides a related example of combining agentic LLM workflows with probabilistic graphical models and numerical Bayesian inference [13]. Bayesian Teaching studies Bayesian updating as a normative and trainable target inside language models [14]. This project instead fits an external controller over observed model behavior motivated by [1].

References

[1] Theodore Papamarkou et al. Position: Agentic AI orchestration should be bayes-consistent, 2026. URL https://doi.org/10.48550/arXiv.2605.00742. Proceedings of the 43rd International Conference on Machine Learning (ICML).

[2] James O. Berger. *Statistical Decision Theory and Bayesian Analysis*. Springer, 1985.

[3] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. CRC Press, 3 edition, 2013.

[4] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.

[5] Deepro Choudhury, Sinead Williamson, Adam Goliński, Ning Miao, Freddie Bickford Smith, Michael Kirchhof, Yizhe Zhang, and Tom Rainforth. BED-LLM: Intelligent information gathering with LLMs and bayesian experimental design. In *International Conference on Learning Representations (ICLR)*, 2026. doi: 10.48550/arXiv.2508.21184. URL https://doi.org/10.48550/arXiv.2508.21184.

[6] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann series in representation and reasoning. Elsevier Science, 1988.

[7] Aki Vehtari, Andrew Gelman, and Jonah Gabry. Practical bayesian model evaluation using leave-one-out cross-validation and WAIC. *Statistics and Computing*, 27(5):1413–1432, 2017.

[8] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[9] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[10] Khurram Yamin, Jingjing Tang, Santiago Cortes-Gomez, Amit Sharma, Eric Horvitz, and Bryan Wilder. When agents say one thing and do another: Validating elicited beliefs from LLMs, 2026. URL https://doi.org/10.48550/arXiv.2602.06286. arXiv preprint.

[11] Chacha Chen, Matthew Jörke, Adam Goliński, Masha Fedzechkina, Guillermo Sapiro, Sinead Williamson, and Nicholas Foti. LLMs are not (consistently) bayesian: Quantifying internal (in)consistencies of LLMs' probabilistic beliefs, 2026. URL https://doi.org/10.48550/arXiv.2605.06915. arXiv preprint.

[12] Zihuiwen Ye, Lukas Aichberger, Michael Kirchhof, Sinead Williamson, Luca Zappella, Yarin Gal, Arno Blaas, and Adam Goliński. Uncertainty quantification for LLM function-calling, 2026. URL https://doi.org/10.48550/arXiv.2604.22985. arXiv preprint.

[13] Hengguan Huang, Xing Shen, Songtao Wang, Lingfa Meng, Dianbo Liu, David Alejandro Duchene, Hao Wang, and Samir Bhatt. BayesAgent: Bayesian agentic reasoning under

uncertainty via verbalized probabilistic graphical modeling. In *AAAI Conference on Artificial Intelligence*, 2026. doi: 10.48550/arXiv.2406.05516. URL https://doi.org/10.48550/arXiv.2406.05516.

[14] Linlu Qiu, Fei Sha, Kelsey Allen, Yoon Kim, Tal Linzen, and Sjoerd van Steenkiste. Bayesian teaching enables probabilistic reasoning in large language models, 2025. URL https://doi.org/10.48550/arXiv.2503.17523. arXiv preprint.